Facultad de Ingeniería
Departamento ingeniería en sistemas
Lenguajes formales y autómatas
Sección 02
Docente: José Abelardo Mendez Pú

Universidad Rafael Landívar
Tradición Jesuita en Guatemala

Proyecto 2
Calculadora algebraica

Javier André Huertas Bravo - 1505825

Quetzaltenango, 30 de octubre de 2,026

1. Definición formal de la Gramática Libre de Contexto (GLC)

Para garantizar él respeto a la jerarquía de operaciones y la asociatividad requerida, la Gramática Libre de Contexto se define formalmente como la cuádrupla $G = (V, \Sigma, R, S)$:

- Conjunto de variables (No terminales): $V = \{E, T, F, P\}$
- Alfabeto de símbolos terminales: $\Sigma = \{+. -, *, /, `, (,), num\}$
- Símbolo inicial: $S = E$
- Conjunto de reglas de producción: (R): $E \ -> \ E \ + \ T // E \ - \ T \ // \ T$

$$T \ -> \ T \ * \ F \ // \ T / F \ // \ F$$

Justificación de Precedencia y Asociatividad en la GLC

Nivel 1 (Suma y resta): Él simbolo E (Expresión) es recursivo por la izquierda, lo que fuerza la asociatividad a la izquierda para + y -.
Nivel 2 (Multiplicacion y division): Él símbolo T (término) también presenta recursión por la izquierda, garantizando asociatividad a la izquierda para * y /
Nivel 3 (Potencia): Él símbolo F (factor) utiliza recursión por la derecha, lo que modela la asociatividad a la derecha del operador (Ej: 2^ 3^ 2 = 512).
NIvel 4 (Paréntesis y Operandos): Él símbolo P (primario) encapsula expresiones entre paréntesis (E) y constantes numéricas.

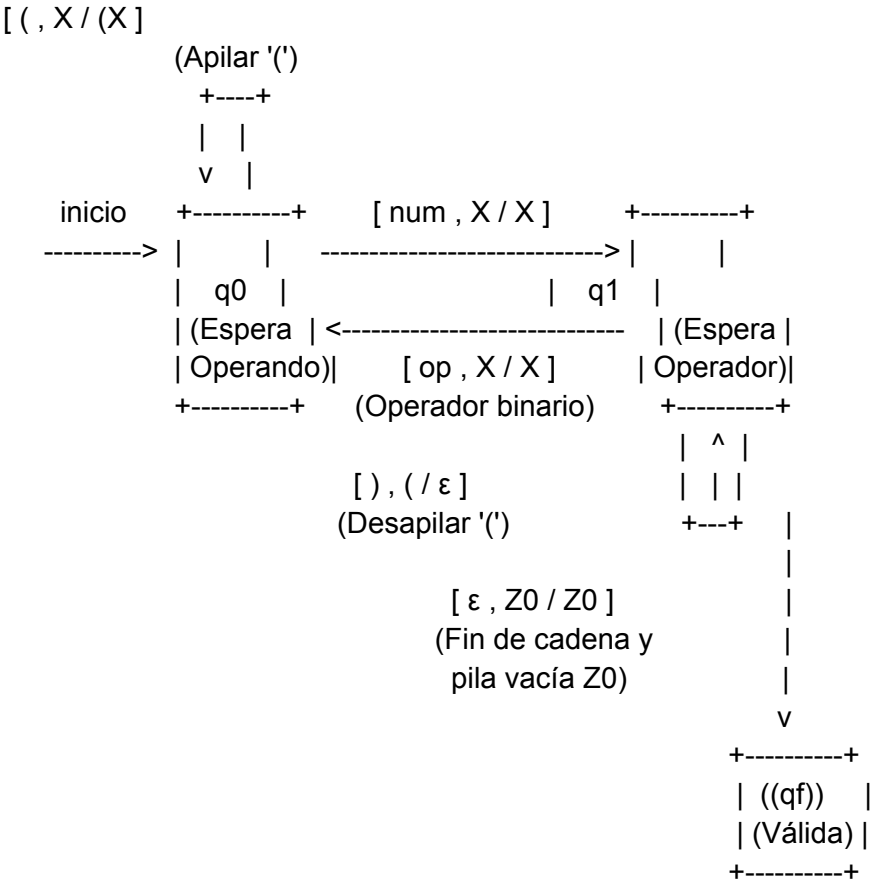
2. Diseño y Diagramas del Autómata de Pila (PDA)

Para validar simultáneamente él balanceo de paréntesis mediante una estructura LIFO y la ubicación sintáctica correcta de operadores y operandos, se diseña un autómata de Pila Determinista $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$:

- **Estados (Q):**
  - q_0: Estado inicial y de espera de operando (inicio de expresión, tras un operador binario o tras abrir paréntesis (().
  - q_1: Estado de operando leído (tras leer un número num o tras cerrar un paréntesis) válido).
  - q_{err}: Estado sumidero de error sintáctico o de desbalance de pila.
  - q_f: Estado de aceptación final.

Alfabeto de entrada (Σ): $\{num, op, (,)\}$ donde $op \ \varepsilon \{+, -, *, /, \hat{} \}$
Alfabeto de la pila (Γ): $\{Z_0, (\}$, donde Z_0 marca el fondo de la pila.

Diagrama de Estados del autómata de pila



(Nota: $X \in \{Z_0, (\}$ representa cualquier símbolo en el tope de la pila. Cualquier transición no definida —como recibir un operador op o) estando en q_0, recibir) con Z_0 en el tope, o finalizar la cadena con (en la pila— envía al autómata al estado de rechazo con un mensaje específico).

Diagrama en formato Mermaid

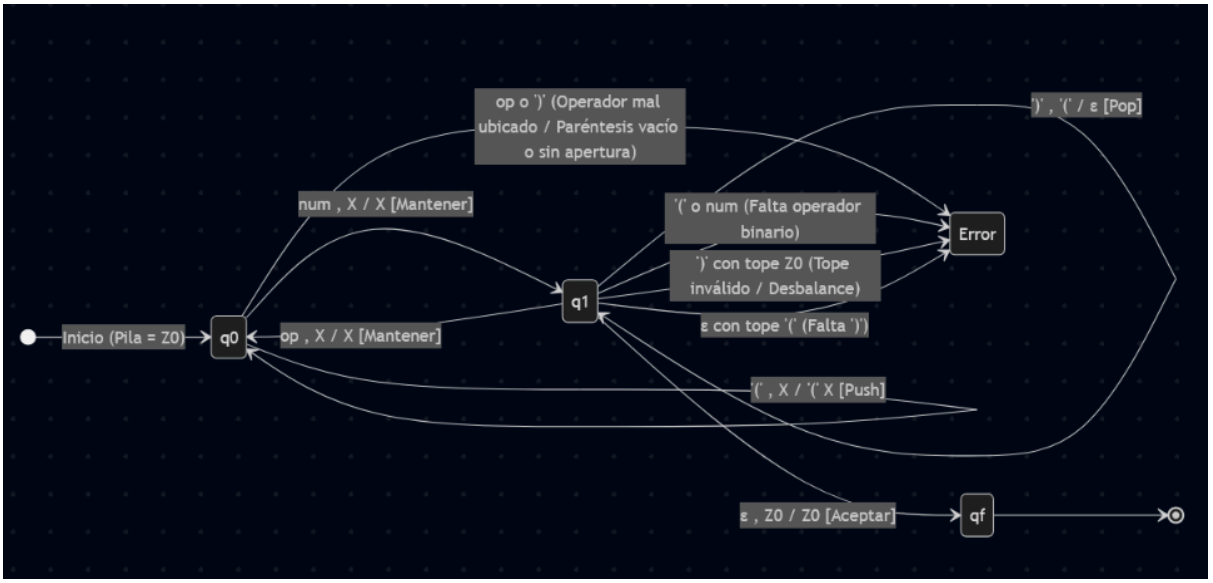


Tabla de Reglas de Transición (δ)

Estado actual	Entrada	Topa de pila	Estado siguiente	Operación en pila	Significado
q_0	(	Z_0 o (	q_0	Push('(') ($\rightarrow (X)$	Abre agrupación y apila (
q_0	num	Z_0 o (	q_1	Ninguna	Operando válido leído
q_0	op	Cualquiera	q_{err}	Error	Operador mal ubicado (ej. 3 + * 5 o + 3)
q_0	)	Z_0	q_{err}	Error	Tope inválido / Desbalance (ej.)2 + 3( )
q_0	)	(	q_{err}	Error	Paréntesis vacío ( ) o falta operando antes de)
q_1	op	Z_0 o (	q_0	Ninguna	Operador binario válido; espera siguiente operando
q_1	)	(	q_1	Pop ()	Cierra paréntesis emparejado correcto
q_1	)	Z_0	q_{err}	Error	Tope inválido / Cierre) sin apertura previa
q_1	ε (Fin)	Z_0	q_f	Aceptar	Expresión válida y pila balanceada
q_1	ε (Fin)	(	q_{err}	Error	Falta) para cerrar paréntesis abiertos

3. Salida y diagramas AST de los casos de prueba

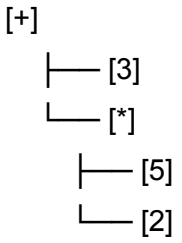
Él acceso aleatorio evita recorrer secuencialmente él archivo desde él principio. Como todos los registros miden exactamente él tamaño récord que son 100 bytes, la posición física de cualquier contacto se calcula multiplicando su número de registro por 100.

Él método f.seek() mueve él puntero de lectura/escritura del sistema operativo directamente al byte calculado, mientras que f.tell() retorna la ubicación actual en bytes para verificar la operación y mostrarla en la barra de estado de la interfaz

Cadena de entrada	Resultado obtenido	Pila / Balanceo	Estado sintáctico	Detalle / Observación
3 + 5 * 2	13	Correcto	Válida	*Se evalúa antes de +
(3 + 5)* 2	16	Correcto	Válida	Paréntesis altera la jerarquía
2 ^ 3 ^ 2	512	Correcto	Válida	Asociatividad derecha
((4 + 2) * (10 - 5))	30	Correcto	Válida	Paréntesis anidados balanceados
(3 + 5 * 2	N/A	Falta)	Error sintáctico	Quedó 1 paréntesis sin cerrar en la pila
3 + * 5	N/A	Correcto	Error sintáctico	Operador * mal ubicado tras + (estado q_0)
) 2 + 3 (	N/A	Tope invalido	Error sintáctico	Se encontró) con tope de pila Z_0
10 / (5 - 5)	Alerta	Correcto	Válida	Captura de división de error por el 0

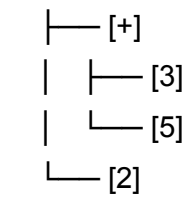
4. Diagramas de los AST generados (Casos válidos)

Caso 1: 3 + 5 * 2, resultado:13



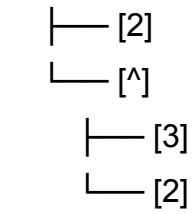
Recorrido postorden: $3 \rightarrow 5 \rightarrow 2 \rightarrow (5 * 2 = 10) \rightarrow (3 + 10 = 13).$

Caso 2: (3+5) * 2, resultado: 16



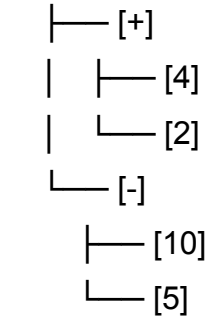
Recorrido postorden: $3 \rightarrow 5 \rightarrow (3 + 5 = 8) \rightarrow 2 \rightarrow (8 * 2 = 16).$

Caso 3: 2 ^ 3 ^ 2 resultado: 512



Recorrido postorden: $2 \rightarrow 3 \rightarrow 2 \rightarrow (3\hat{}\ 2 = 9) \rightarrow (2\hat{}\ 9 = 512)$.

Caso 4: ((4+2) * (10-5)), resultado: 30

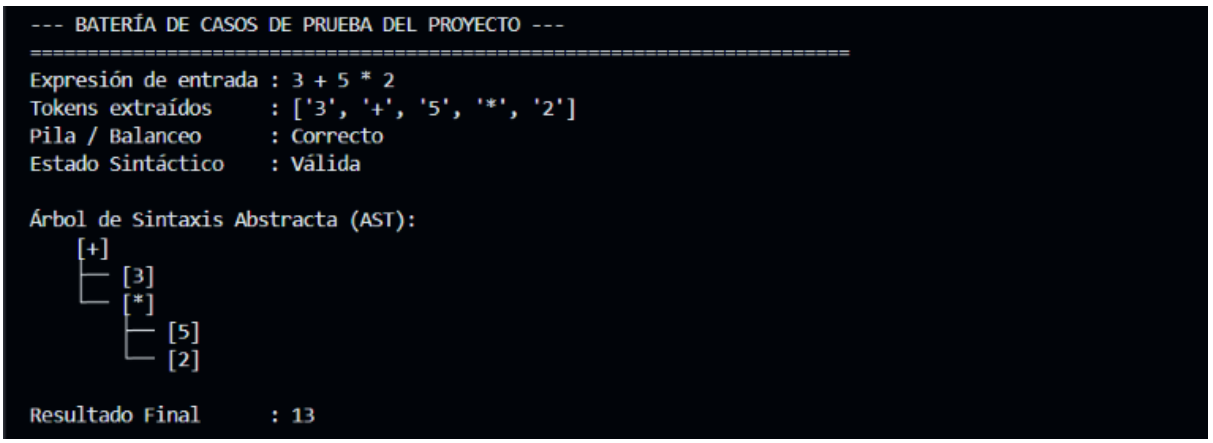


Recorrido postorden:

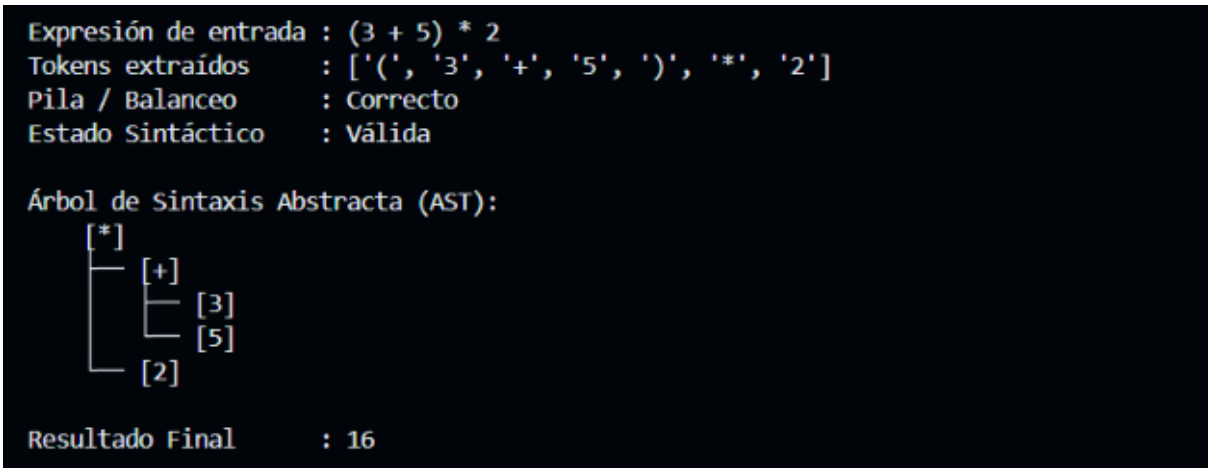
$4 \rightarrow 2 \rightarrow (4 + 2 = 6) \rightarrow 10 \rightarrow 5 \rightarrow (10 - 5 = 5) \rightarrow (6 * 5 = 30).$

5. Capturas de pantalla programa funcionando

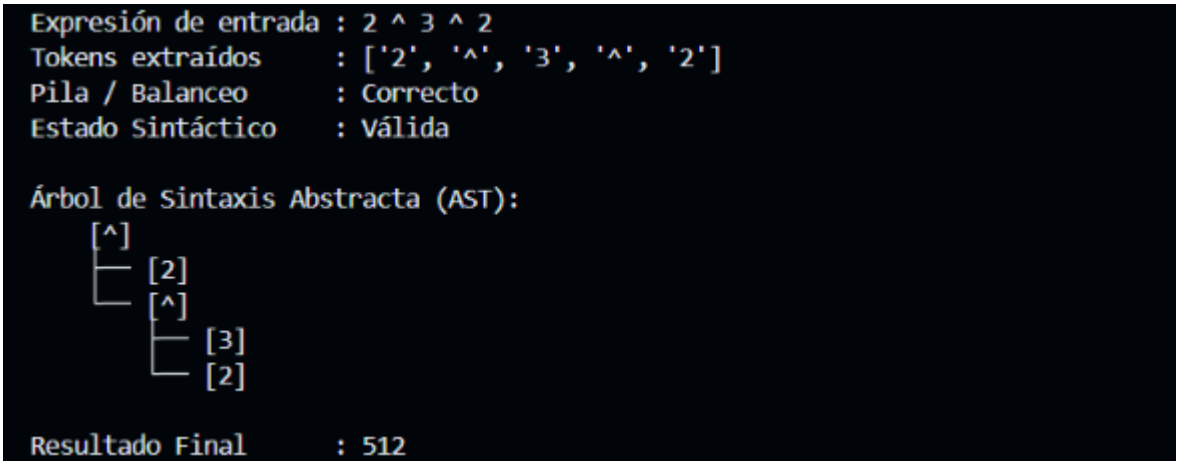
Ejemplo 1



Ejemplo 2



Ejemplo 3



Ejemplo 4

```
Expresión de entrada : ((4 + 2) * (10 - 5))
Tokens extraídos     : ['(', '(', '4', '+', '2', ')', '*', '(', '10', '-', '5', ')', ')']
Pila / Balanceo      : Correcto
Estado Sintáctico    : Válida

Árbol de Sintaxis Abstracta (AST):
    [*]
    ├── [+]
    │   ├── [4]
    │   └── [2]
    └── [-]
        ├── [10]
        └── [5]

Resultado Final      : 30
```

Errores sintacticos

```
Expresión de entrada : (3 + 5 * 2
Tokens extraídos     : ['(', '3', '+', '5', '*', '2']
Pila / Balanceo      : Falta )
Estado Sintáctico    : Error Sintáctico
Detalle del fallo    : Quedaron 1 paréntesis '(' sin cerrar en la pila.
Resultado Esperado   : N/A
======================================================================
Expresión de entrada : 3 + * 5
Tokens extraídos     : ['3', '+', '*', '5']
Pila / Balanceo      : Correcto
Estado Sintáctico    : Error Sintáctico
Detalle del fallo    : Operador '*' mal ubicado en posición 3 (se esperaba un operando o '('.
Resultado Esperado   : N/A
======================================================================
Expresión de entrada : )2 + 3(
Tokens extraídos     : [')', '2', '+', '3', '(']
Pila / Balanceo      : Tope inválido / Desbalance
Estado Sintáctico    : Error Sintáctico
Detalle del fallo    : Paréntesis de cierre ')' inválido al inicio o sin apertura previa (posición 1).
Resultado Esperado   : N/A
```

División entre 0

```
Expresión de entrada : 10 / (5 - 5)
Tokens extraídos     : ['10', '/', '(', '5', '-', '5', ')']
Pila / Balanceo      : Correcto
Estado Sintáctico    : Válida

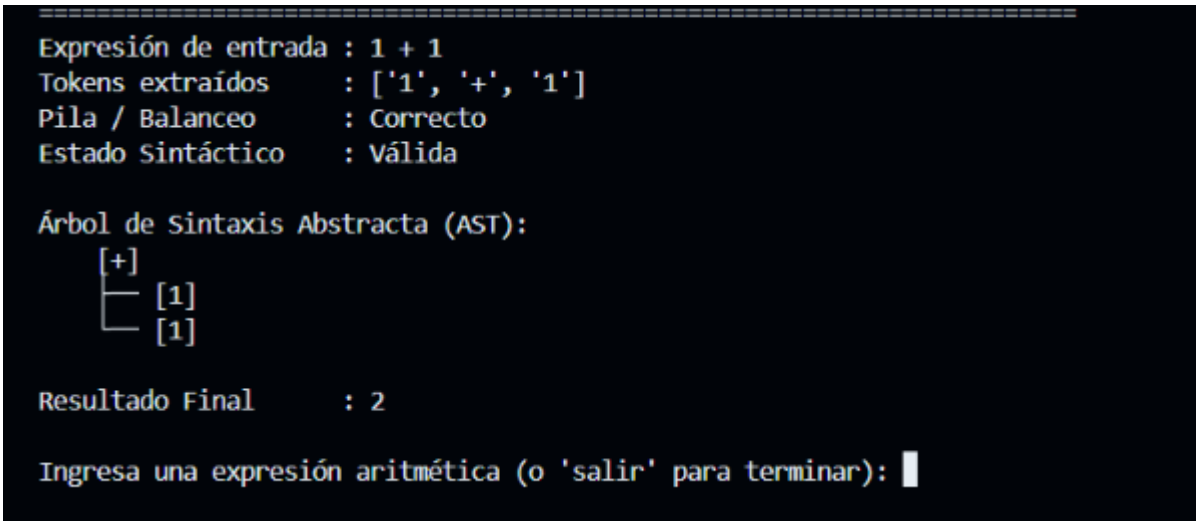
Árbol de Sintaxis Abstracta (AST):
    [/]
    ├── [10]
    └── [-]
        ├── [5]
        └── [5]

Alerta de Evaluación : Error Matemático: División entre cero no permitida.
```

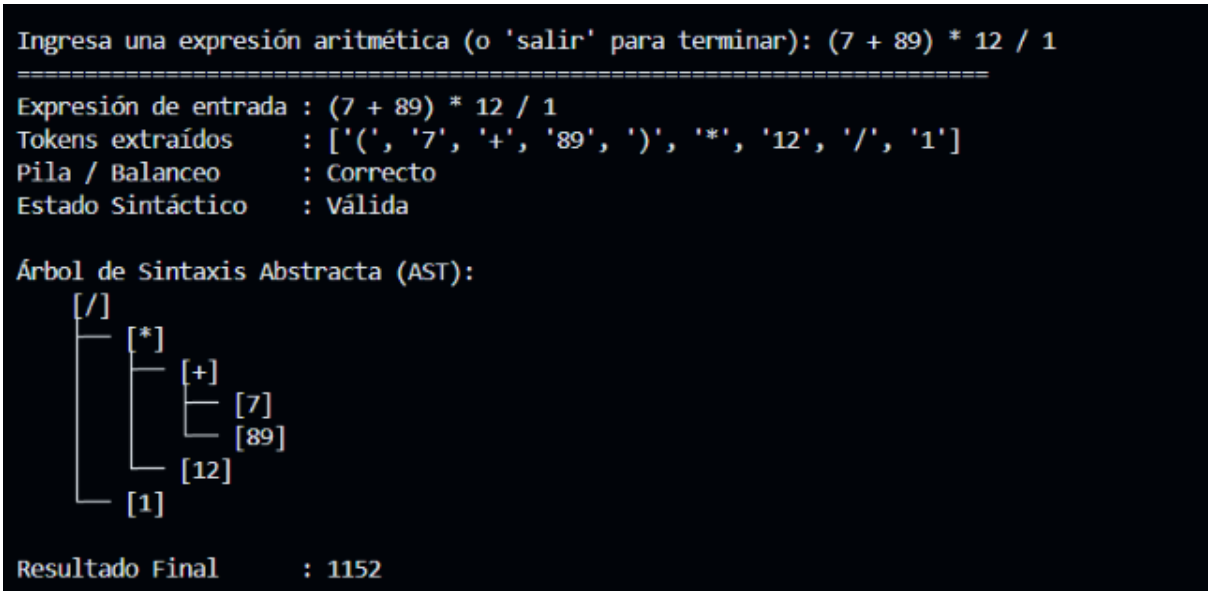
Prueba extras (Error de sintaxis)

```
Ingresa una expresión aritmética (o 'salir' para terminar): +
======================================================================
Expresión de entrada : +
Tokens extraídos     : ['+']
Pila / Balanceo      : Correcto
Estado Sintáctico    : Error Sintáctico
Detalle del fallo    : Operador '+' mal ubicado en posición 1 (se esperaba un operando o '('.
Resultado Esperado   : N/A
```

Ejemplo 7



Ejemplo 8



6. Link del video de demostración

https://youtu.be/Z6IFmjq4tFk